Understanding MCP

Model Context Protocol: a plain-language guide with a working example

1. The Core Idea

In the agentic AI guide, **tools** were the hands of an agent: functions like `search_flights()`. But every developer wrote those tools differently, and every AI app connected to them in its own custom way.

MCP (Model Context Protocol) is an open standard that defines *one common way* for AI applications to connect to tools and data. It was introduced by Anthropic and is now supported by many AI apps and services.

> **Analogy: USB-C for AI.** Before USB-C, every device had its own charger and cable. MCP is the common plug: build a tool once as an MCP server, and any MCP-compatible AI app can use it.

The problem MCP solves

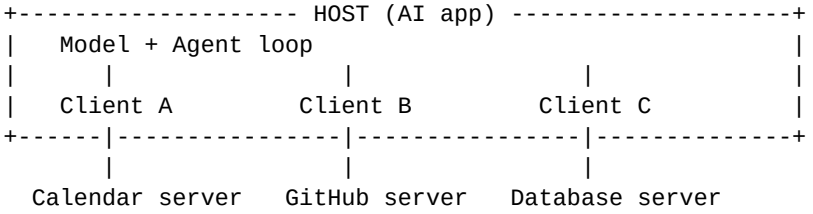
Without a standard, connecting 3 AI apps to 4 services (Gmail, Calendar, a database, GitHub) means writing up to **3 x 4 = 12** custom integrations. With MCP, each service builds one server and each app builds one client, so you need only **3 + 4 = 7** pieces, and every new app or service plugs in for free.

Important: MCP is not an agent

MCP does not think or decide anything. It only standardizes *how tools and data are exposed and called*. The agent (model + loop) is still the thing that decides. In our four-part picture, MCP lives inside the **Tools** part.

2. The Three Roles: Host, Client, Server

Role	What it is	Example
Host	The AI application the user interacts with. It contains the model and the agent loop.	Claude Desktop, an IDE, your own agent app
Client	A connector inside the host. One client talks to exactly one server.	Created by the host automatically
Server	A small program that exposes tools, data and prompts in the MCP format.	Calendar server, GitHub server, database server



The host can connect to many servers at once. The model sees all their tools together as one combined menu.

3. What a Server Can Offer: The Three Primitives

Primitive	Who controls it	What it is	Example
Tools	The **model** decides when to call	Actions that do something or compute something	search_flights, send_email
Resources	The **application** decides what to load	Read-only data used as context	A file, a database record, user preferences
Prompts	The **user** picks them	Reusable templates for common tasks	"Plan a trip to X"

Tools are the most used. They map directly to the 'hands' of an agent. **Resources** are like attachments the app can hand to the model. **Prompts** are shortcuts a user can trigger, like slash commands.

Features the client side can offer back

- **Sampling:** the server can ask the host's model to generate text (the server borrows the brain).
- **Roots:** the client tells the server which folders or areas it is allowed to work in.
- **Elicitation:** the server can ask the user for missing information mid-task.

4. How It Talks: Transport and Messages

Messages: MCP uses **JSON-RPC 2.0**, a simple format where the client sends a request with a method name and the server sends back a result.

Transports (how the messages travel):

- **stdio:** the host starts the server as a local subprocess and they talk through standard input/output. Simple, fast, used for local tools.
- **Streamable HTTP:** the server runs remotely and is reached over HTTP. Used for cloud services, and supports authentication.

What a real exchange looks like

```
Client -> Server:  {"jsonrpc":"2.0","id":1,"method":"tools/list"}

Server -> Client:  {"jsonrpc":"2.0","id":1,"result":{"tools":[
                     {"name":"search_flights",
                      "description":"Find flights between two cities...",
                      "inputSchema":{ ... }}]}}

Client -> Server:  {"jsonrpc":"2.0","id":2,"method":"tools/call",
                    "params":{"name":"search_flights",
                      "arguments":{"origin":"Hyderabad","destination":"Delhi","date":"2026-11-06"}}}
```

You rarely write this by hand. The MCP SDKs do it for you, but it helps to know that underneath it is just 'list the tools' and 'call this tool'.

5. The Lifecycle of a Connection

Step	What happens
1. Connect	The host launches or reaches the server over a transport (stdio or HTTP).

Step	What happens
2. Initialize	Client and server exchange protocol version and declare their capabilities (what each supports).
3. Discover	Client asks: `tools/list`, `resources/list`, `prompts/list`. Tools are found at runtime, nothing is hard-coded.
4. Use	The model picks a tool; the client sends `tools/call`; the server runs it and returns the result.
5. Close	The connection ends and the server process stops.

Why discovery matters: if the server's developer adds a new tool tomorrow, the agent finds it automatically next time it connects. You do not change the agent's code.

6. Agent With Plain Tools vs Agent With MCP

	Plain function tools (earlier example)	MCP tools
Where tools live	Inside your agent's code	In a separate server program
How the agent learns them	You hard-code the tool descriptions	Discovered at runtime with tools/list
Reuse	Only your app can use them	Any MCP-compatible app can use them
Adding a tool	Edit and redeploy the agent	Add it to the server; agents see it automatically
The agent loop	Think, act, observe	**Exactly the same**

The loop does not change at all. Only the line that *runs the tool* changes: instead of calling a local Python function, the agent calls `session.call_tool(name, arguments)` on the MCP session.

7. The Working Example

Two files are provided: `mcp_flight_server.py` and `mcp_agent_client.py`.

The server (mcp_flight_server.py)

It knows nothing about Claude. It only declares what it offers, using decorators:

```
server = Server("flight-assistant")

@server.tool()
def search_flights(origin: str, destination: str, date: str) -> list[dict]:
    """Find flights between two cities on a date (YYYY-MM-DD)."""
    ...

@server.tool()
def add_calendar_event(title: str, date: str, time: str) -> dict: ...

@server.resource("user://preferences")
def user_preferences() -> str: ...

@server.prompt()
def plan_trip(destination: str) -> str: ...
```

The function name, type hints and docstring are automatically turned into the tool description the model reads.

The client + agent (mcp_agent_client.py)

```
async with stdio_client(params) as (read, write):      # 1. launch server, connect
    async with ClientSession(read, write) as session:
        await session.initialize()                      # 2. handshake
        tools = (await session.list_tools()).tools      # 3. discover tools
        ...
        for step in range(MAX_STEPS):                   # 4. same agent loop
            response = llm.messages.create(tools=tool_specs, messages=messages)
            if response.stop_reason != "tool_use": break
            for block in response.content:
                if block.type == "tool_use":
                    out = await session.call_tool(block.name, block.input)   # MCP call
            ...append results to messages...
```

How to run

- Install: `pip install anthropic mcp`
- Put both files in the same folder.
- Without an API key (scripted fake model): `python mcp_agent_client.py --demo`
- With Claude: set `ANTHROPIC_API_KEY`, then `python mcp_agent_client.py`

The client launches the server by itself, so you do not start the server separately.

8. Full Flow, Step by Step

Same flight goal, now with each part labeled:

#	Who	What happens
1	Host (client)	Launches the server; sends initialize; handshake completes
2	Client	Calls tools/list; server replies with search_flights and add_calendar_event
3	Host	Gives the model the goal plus the discovered tool menu
4	Model	Replies: call search_flights for Delhi, next Friday
5	Client	Sends tools/call to the server
6	Server	Runs the real function and returns the flight list
7	Memory	Result added to the conversation history
8	Model	Picks the cheapest and asks for add_calendar_event
9	Client / Server	tools/call again; server creates the event
10	Model	Gives the final answer; loop stops

9. Security and Good Practice

- **Trust only servers you trust.** A server's tool descriptions and results go into the model's context, so a malicious server can try to steer the model (prompt injection).
- **Ask before risky actions.** Hosts usually show a confirmation before the model sends email, deletes files or spends money.
- **Least privilege.** Give each server only the access it needs (for example, one folder, not the whole disk).
- **Authentication for remote servers.** Remote (HTTP) servers should require proper sign-in rather than being open.
- **Never print to stdout in a stdio server.** Stdout is the protocol channel; use stderr for logs.

10. Summary

- **Agent** = model + tools + memory + loop (the earlier guide).
- **MCP** = a standard way to plug tools and data into agents.
- **Host** runs the agent; **client** connects; **server** exposes tools, resources and prompts.
- Tools are **discovered at runtime**, so servers are reusable across apps.
- The agent loop stays exactly the same; only *where the tool runs* changes.